Lekcja 3: Podstawy Pythona – Część 1

`#lekcja` `#python` `#podstawy` `#zmienne` `#typy-danych`

Na tej lekcji położymy fundamenty naszej wiedzy o Pythonie. Omówimy podstawowe elementy składowe języka: jak przechowywać dane w zmiennych, jakie typy danych istnieją, jak wykonywać operacje i pracować z tekstem. Zwrócimy szczególną uwagę na "dobre maniery" w pisaniu kodu oraz kluczowe pojęcie mutowalności danych.

1. Czym jest PEP?

> **Info**
>
> PEP — to skrót od Python Enhancement Proposal (Propozycja Ulepszenia Pythona). Jest to oficjalny dokument, który proponuje nowe funkcje, opisuje standardy lub najlepsze praktyki dla języka Python.

Każdy PEP ma swój numer. Najważniejszy dla początkujących to:

📜 PEP 8 – Przewodnik po stylu kodu w Pythonie

PEP 8 to nie prawo, a raczej zbiór zaleceń, "dobrych manier", których przestrzega większość programistów na świecie. Jego celem jest uczynienie kodu czystym, czytelnym i jednolitym, aby każdy inny programista mógł go łatwo zrozumieć.

Kluczowe zasady z PEP 8:

- **Wcięcia:** Używaj **4 spacji** na każdy poziom wcięcia. Nie używaj tabulatorów.
- **Długość linii:** Ograniczaj długość linii do **79-99 znaków**.
- **Spacje:**
  - Stawiaj spacje wokół operatorów (`x = y + 1` , a nie `x=y+1`).
  - Nie stawiaj spacji zaraz po otwierającym i przed zamykającym nawiasem (`print(x)` , a nie `print( x )`).
- **Nazewnictwo:**
  - `snake_case` (małe_litery_z_podkreśleniami) dla zmiennych i funkcji.
  - `PascalCase` (WielkieLiteryJakWielbłąd) dla klas.
- **Puste linie:** Używaj pustych linii do oddzielania logicznych bloków kodu.

> **Analogia**

PEP 8 jest jak zasady gramatyki i interpunkcji w języku. Bez nich tekst da się zrozumieć, ale z nimi staje się jasny, profesjonalny i przyjemny w czytaniu. Większość IDE (w tym VSCode) może automatycznie sprawdzać zgodność Twojego kodu z PEP 8.

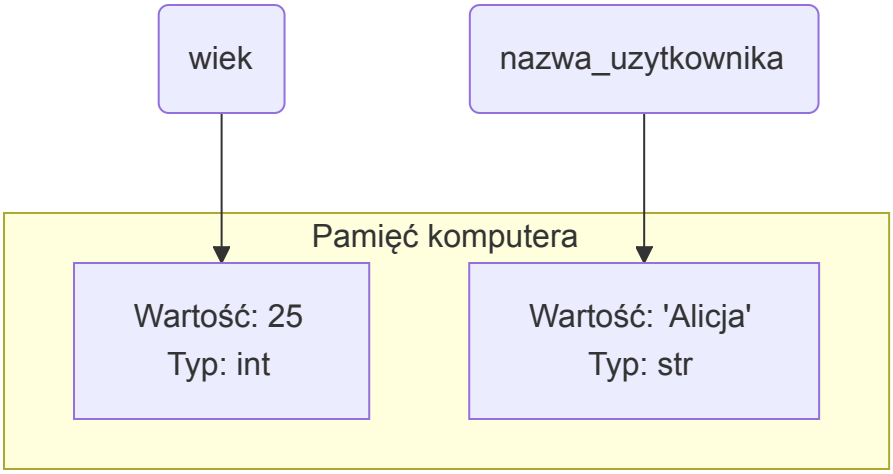
2. Zmienne

Definition

Zmienna — to nazwany obszar w pamięci komputera, który przechowuje określoną wartość. Mówiąc prościej, jest to nazwana komórka lub etykieta dla danych.

```
# Tworzymy zmienną 'wiek' i umieszczamy w niej wartość 25
wiek = 25

# Tworzymy zmienną 'nazwa_uzytkownika' i przypisujemy jej wartość tekstową "Alicja"
nazwa_uzytkownika = "Alicja"
```



Zasady nazewnictwa zmiennych:

- Nazwa może zawierać litery (a-z, A-Z), cyfry (0-9) i znak podkreślenia (`_`).
- Nazwa **nie może** zaczynać się od cyfry.
- Nazwy są wrażliwe na wielkość liter (`wiek` i `Wiek` — to dwie różne zmienne).
- Nie można używać **słów kluczowych (zarezerwowanych)** języka jako nazw.

Słowa kluczowe Pythona

Są to słowa, które mają specjalne znaczenie w języku, na przykład: if, for, class, def, while, True, False, None.

Aby zobaczyć pełną listę, można wykonać kod: import keyword; print(keyword.kwlist)

3. Typy danych

Python to język z **dynamicznym typowaniem**. Oznacza to, że nie musisz określać typu zmiennej; Python określa go automatycznie na podstawie przypisanej wartości.

Podstawowe wbudowane typy:

Typ	Nazwa	Przykład	Opis
`int`	Integer	`x = 100`	Liczba całkowita (dodatnia lub ujemna).
`float`	Floating-Point	`pi = 3.14`	Liczba zmiennoprzecinkowa (ułamkowa).
`str`	String	`msg = "Witaj"`	Tekst, czyli sekwencja znaków.
`bool`	Boolean	`jest_aktywny = True`	Typ logiczny: `True` (prawda) lub `False` (fałsz).
`list`	Lista	`elementy = [1, "a", 3.0]`	Uporządkowana, **mutowalna** (zmienna) kolekcja elementów.
`tuple`	Krotka	`punkt = (10, 20)`	Uporządkowana, **niemutowalna** (niezmienna) kolekcja elementów.
`dict`	Słownik	`uzytkownik = {"klucz": "wartość"}`	Nieuporządkowana (w starszych wersjach) kolekcja par "klucz:wartość".
`set`	Zbiór	`unikalne = {1, 2, 3}`	Nieuporządkowana kolekcja **unikalnych** elementów.
`NoneType`	None	`wynik = None`	Specjalny typ oznaczający "brak wartości".

Sprawdzanie i konwersja typów

- Funkcja `type()` pozwala poznać typ zmiennej.
- Funkcje `int()`, `float()`, `str()`, `list()` pozwalają jawnie konwertować jeden typ na inny (rzutowanie).

```
wartosc_str = "123"
print(type(wartosc_str))  # <class 'str'>
```

```python
# Konwertujemy string na liczbę
wartosc_int = int(wartosc_str)
print(type(wartosc_int))  # <class 'int'>

wynik = wartosc_int + 10 # Teraz można wykonywać operacje arytmetyczne
print(wynik)             # 133
```

4. Operatory w Pythonie

Operatory arytmetyczne

Operator	Nazwa	Przykład	Wynik
`+`	Dodawanie	`10 + 5`	`15`
`-`	Odejmowanie	`10 - 5`	`5`
`*`	Mnożenie	`10 * 5`	`50`
`/`	Dzielenie	`10 / 4`	`2.5`
`//`	Dzielenie całkowite	`10 // 4`	`2`
`%`	Reszta z dzielenia	`10 % 4`	`2` (10 = 2*4 + **2**)
`**`	Potęgowanie	`2 ** 3`	`8`

Operatory porównania (zwracają `True` lub `False`)

Operator	Znaczenie
`==`	Równy
`!=`	Różny
`>`	Większy niż
`<`	Mniejszy niż
`>=`	Większy lub równy
`<=`	Mniejszy lub równy

Operatory logiczne

Operator	Znaczenie	Przykład

`and`	Logiczne "I" (koniunkcja)	`(wiek > 18) and (ma_prawo_jazdy == True)`
`or`	Logiczne "LUB" (alternatywa)	`jest_weekend or ma_dzien_wolny`
`not`	Logiczne "NIE" (negacja)	`not jest_aktywny`

5. Łańcuchy znaków (`str`)

Łańcuchy znaków (stringi) to jeden z najczęściej używanych typów danych. Są to sekwencje znaków ujęte w pojedyncze (`'...'`) lub podwójne (`"..."`) cudzysłowy.

Przydatne operacje i metody

```
tekst = "  witaj python!  "

# Długość łańcucha
print(len(tekst)) # 17

# Dostęp przez indeks (numeracja od 0)
print(tekst[8]) # 'p'

# Wycinki (slicing)
print(tekst[8:14]) # 'python' (znaki od 8 do 13)

# Usuwanie białych znaków na początku i na końcu
czysty_tekst = tekst.strip()
print(czysty_tekst) # 'witaj python!'

# Zmiana wielkości liter
print(czysty_tekst.upper()) # 'WITAJ PYTHON!'
print(czysty_tekst.capitalize()) # 'Witaj python!'

# Zamiana podciągu
print(czysty_tekst.replace("python", "świecie")) # 'witaj świecie!'

# Sprawdzanie początku i końca łańcucha
print(czysty_tekst.startswith("witaj")) # True
```

Formatowanie łańcuchów: f-stringi

F-stringi (wprowadzone w Pythonie 3.6) to nowoczesny i najwygodniejszy sposób wstawiania zmiennych do tekstu.

```python
imie = "Alex"
wiek = 30

# Stary sposób (konkatenacja)
powitanie_stare = "Witaj, " + imie + ". Masz " + str(wiek) + " lat."

# Nowy, wygodny sposób (f-string)
powitanie_nowe = f"Witaj, {imie}. Masz {wiek} lat."

print(powitanie_nowe)
```

6. Typy mutowalne (Mutable) i niemutowalne (Immutable)

To jedno z najbardziej fundamentalnych pojęć w Pythonie.

Niemutowalne (Immutable)

Wartości tych obiektów **nie można** zmienić po ich utworzeniu. Każda "modyfikacja" w rzeczywistości tworzy **nowy** obiekt w pamięci.

- `int`, `float`, `bool`, `str`, `tuple`

```python
# Stringi są niemutowalne
imie = "Alex"
# imie[0] = "B" # To spowoduje błąd TypeError!

# "Zmiana" stringa tworzy nowy obiekt
nowe_imie = imie.replace("A", "B")
print(imie)      # "Alex" (oryginał się nie zmienił)
print(nowe_imie) # "Blex" (to już jest nowy obiekt)
```

Mutowalne (Mutable)

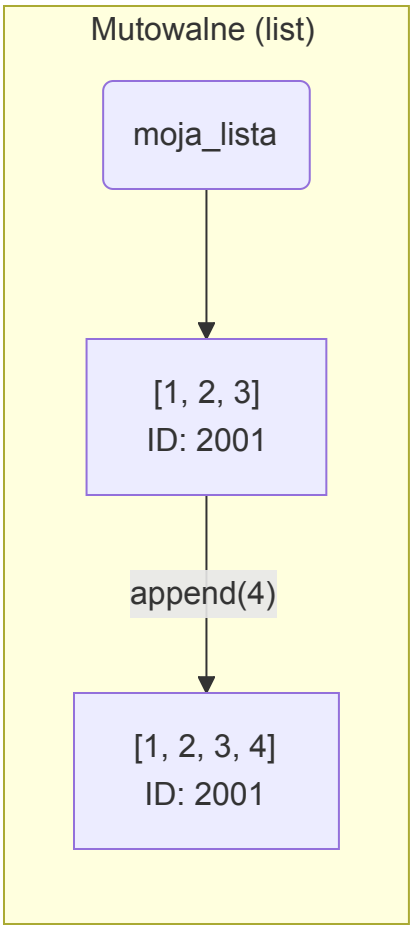
Wartość tych obiektów **można** zmienić po ich utworzeniu, nie tworząc nowego obiektu.

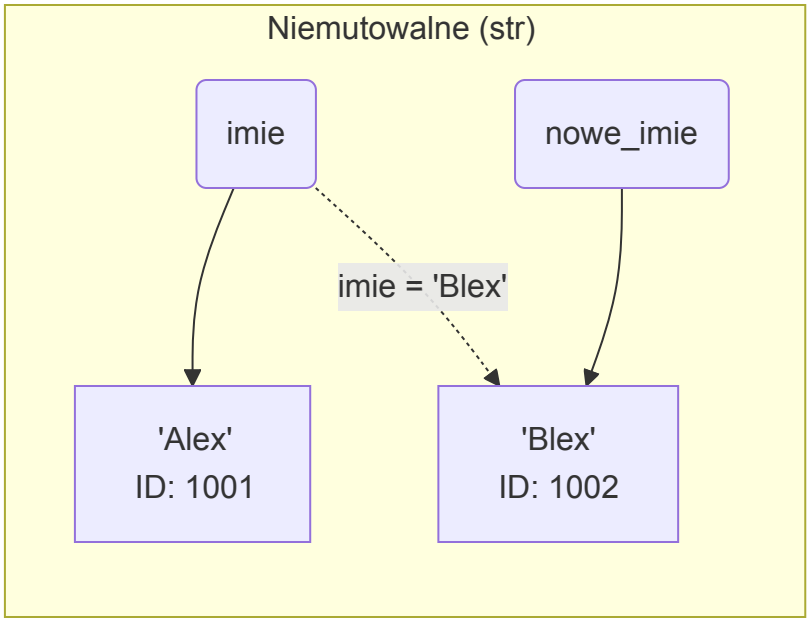
- `list`, `dict`, `set`

```python
# Listy są mutowalne
moja_lista = [1, 2, 3]
print(f"ID listy przed zmianą: {id(moja_lista)}") # id() pokazuje adres
obiektu w pamięci

moja_lista.append(4) # Zmieniamy obiekt "w miejscu"
moja_lista[0] = 99
```

```python
print(moja_lista) # [99, 2, 3, 4]
print(f"ID listy po zmianie: {id(moja_lista)}") # ID pozostało takie samo!
```





Note

Zrozumienie tej różnicy jest krytycznie ważne, zwłaszcza przy przekazywaniu danych do funkcji, ponieważ funkcje mogą modyfikować listy i słowniki przekazane im jako argumenty.

7. Zadania praktyczne

1. **Tworzenie i typowanie:** Utwórz zmienne przechowujące Twoje imię (str), wiek (int), średnią ocen (float) i status studenta (bool). Wyświetl na ekranie wartość i typ każdej zmiennej.
2. **Kalkulator BMI:** Napisz program, który zapyta użytkownika o jego wagę w kilogramach i wzrost w metrach. Oblicz i wyświetl wskaźnik masy ciała (BMI) według wzoru: `BMI = waga / (wzrost * wzrost)`.
3. **Analiza stringa:** Utwórz zmienną z łańcuchem znaków `" Python jest super! "`. Wykonaj następujące działania i wyświetl wynik każdego kroku:
   - Usuń zbędne białe znaki na początku i na końcu.

- Przekształć cały ciąg na małe litery.
- Zamień słowo "super" na "świetny".
- Wyświetl na ekranie znak pod indeksem `4`.

4. **Praca z f-stringami:** Poproś użytkownika o jego imię i rok urodzenia. Oblicz jego przybliżony wiek i wyświetl komunikat w formacie: `"Cześć, [Imię]! W 2025 roku będziesz mieć około [Wiek] lat."`
5. **Popraw kod zgodnie z PEP 8:** Poniżej znajduje się fragment kodu. Przepisz go tak, aby był zgodny z zasadami PEP 8.

```
# Kod źródłowy
NazwaUzytkownika="JanKowalski"
wiekUzytkownika=25
if wiekUzytkownika>=18:print(NazwaUzytkownika+' jest dorosły.')
```

6. **Operacje na liście:**
   - Utwórz listę `owoce = ["jabłko", "banan", "wiśnia"]`.
   - Dodaj na koniec listy "pomarańczę".
   - Zmień drugi element ("banan") na "jagodę".
   - Wyświetl końcową listę na ekran.
7. **Niemutowalność krotki:**
   - Utwórz krotkę `punkt = (10, 20, 30)`.
   - Spróbuj zmienić pierwszy element krotki на `15`.
   - Wyjaśnij w komentarzu do kodu, dlaczego wystąpił błąd.
8. **Rzutowanie typów:** Utwórz zmienną `liczba_str = "5.8"`. Przekonwertuj ją najpierw na `float`, a następnie na `int` i wyświetl wyniki obu konwersji. Co zauważyłeś podczas konwersji `float` na `int`?
9. **Bramki logiczne:** Napisz program, który poprosi o dwie wartości logiczne (`True` lub `False`). Niech użytkownik wprowadza `1` dla `True` i `0` dla `False`. Program powinien wyświetlić wyniki operacji `AND` oraz `OR` dla tych dwóch wartości.
10. **Mini-projekt "Formater danych":** Napisz program, który poprosi użytkownika o jego imię i nazwisko w jednej linii (np. " jan kowalski "). Program powinien:
    - Oczyścić zbędne białe znaki.
    - Sprawić, aby każde słowo zaczynało się wielką literą (metoda `.title()`).
    - Wyświetlić sformatowane dane oraz ich długość.